

03_baseline_model

July 15, 2026

1 03 — Phase 1: Train and evaluate the baseline wage model

Purpose: run the full Phase 1 pipeline end-to-end on real data — load, build features, clean, split by occupation, train the monotonic-constrained XGBoost model, and compare it against the naive percentile-only baseline. This is the real version of the checks we already validated on synthetic data in `tests/test_wage_model.py`.

```
[1]: import sys
      from pathlib import Path

      project_root = Path.cwd().parent
      sys.path.insert(0, str(project_root))

      import pandas as pd

      from src.data.onet_loader import (
          load_occupation_data,
          load_essential_skills,
          load_transferable_skills,
          load_job_zones,
          build_full_skill_importance_matrix,
      )
      from src.data.oews_loader import load_national_wages
      from src.features.occupation_features import (
          build_occupation_feature_table,
          build_training_rows,
          clean_training_rows,
      )
      from src.models.wage_model import (
          occupation_train_test_split,
          train_baseline_model,
          evaluate,
          check_monotonicity,
      )

[2]: occ = load_occupation_data()
      skills = load_essential_skills()
      transferable = load_transferable_skills()
```

```

job_zones = load_job_zones()
wages = load_national_wages()

skill_matrix = build_full_skill_importance_matrix(skills, transferable)
feature_table = build_occupation_feature_table(occ, skill_matrix, job_zones,
↪wages)
training_rows = build_training_rows(feature_table, skill_cols=list(skill_matrix.
↪columns))
cleaned_training_rows = clean_training_rows(training_rows,
↪skill_cols=list(skill_matrix.columns))

print(f"\nFinal training set: {cleaned_training_rows.shape}")
print(f"Unique occupations: {cleaned_training_rows['onetsoc_code'].nunique()}")

```

[oews_loader] 7 rows have suppressed wage data - kept as NaN, not dropped.
Dropped 470 rows (94 occupations) missing all skill values, out of 4780 total rows.

No job_zone imputation needed.

Final training set: (4310, 39)

Unique occupations: 862

```

[3]: feature_cols = list(skill_matrix.columns) + ["job_zone", "percentile"]

train_df, test_df = occupation_train_test_split(cleaned_training_rows,
↪test_size=0.2, random_state=42)

print(f"Train occupations: {train_df['onetsoc_code'].nunique()}
↪({len(train_df)} rows)")
print(f"Test occupations: {test_df['onetsoc_code'].nunique()} ({len(test_df)}
↪rows)")
print(
    "No occupation overlap between train/test:",
    set(train_df["onetsoc_code"]).isdisjoint(set(test_df["onetsoc_code"])),
)

```

Train occupations: 689 (3445 rows)

Test occupations: 173 (865 rows)

No occupation overlap between train/test: True

```

[4]: model = train_baseline_model(train_df, feature_cols)
print("Model trained.")

```

Model trained.

1.1 Evaluation: does the model actually earn its complexity?

Comparing against the naive baseline (predict using only the training set's mean log-wage for that percentile level, ignoring occupation features entirely). If the real model doesn't clearly beat this,

skill profile + job_zone aren't adding real value and the approach needs rethinking before Phase 2.

```
[5]: result = evaluate(model, train_df, test_df, feature_cols)
      print(result.summary())
```

```
Model RMSE (log-wage):  0.2690
Naive RMSE (log-wage):  0.4560
Model MAE (dollars):    $19,586
Naive MAE (dollars):    $31,943
Model improvement over naive: 38.7%
Test set: 865 rows across 173 held-out occupations
```

1.2 Monotonicity check on real held-out occupations

For every held-out test occupation, predicted wage must never decrease as percentile increases (10th <= 25th <= 50th <= 75th <= 90th). This should hold by construction thanks to the monotonic constraint — confirming it actually does on real data, not just the synthetic test suite.

```
[6]: violations = 0
      violation_codes = []
      for code in test_df["onetsoc_code"].unique():
          row = test_df[test_df["onetsoc_code"] == code].iloc[0]
          feature_values = {col: row[col] for col in feature_cols if col != "percentile"}
          if not check_monotonicity(model, feature_values, feature_cols):
              violations += 1
              violation_codes.append(code)

      print(f"Occupations with non-monotonic predictions: {violations} / {test_df['onetsoc_code'].nunique()}")
      if violations == 0:
          print("All clean - monotonic constraint holds across every held-out occupation.")
      else:
          print(f"Violating occupation codes: {violation_codes}")
```

```
Occupations with non-monotonic predictions: 0 / 173
All clean - monotonic constraint holds across every held-out occupation.
```

1.3 Spot-check: estimating wages for occupations with NO direct BLS match

This is the real value-add of having a model instead of a lookup table — estimating wages for the ~60 crosswalk-unmatched occupations from Phase 0, by interpolating from their skill profile and job_zone alone.

```
[7]: import numpy as np
```

```

no_wage_match = feature_table[feature_table["a_median"].isna() &
↳feature_table[list(skill_matrix.columns)].notna().all(axis=1)]
print(f"Occupations with skill data but NO BLS wage match:
↳{len(no_wage_match)}")

if len(no_wage_match) > 0:
    sample = no_wage_match.head(5).copy()
    for onetsoc_code, row in sample.iterrows():
        feature_values = {col: row[col] for col in feature_cols if col !=
↳"percentile"}
        rows = pd.DataFrame([feature_values] * 5)
        rows["percentile"] = [10, 25, 50, 75, 90]
        preds_log = model.predict(rows[feature_cols])
        preds_dollars = np.exp(preds_log)
        print(f"\n{row['title']} ({onetsoc_code}):")
        print(f"    Estimated wage range: ${preds_dollars[0]:,.0f} (10th) to
↳${preds_dollars[4]:,.0f} (90th), median ${preds_dollars[2]:,.0f}")

```

Occupations with skill data but NO BLS wage match: 32

Buyers and Purchasing Agents, Farm Products (13-1021.00):

Estimated wage range: \$44,499 (10th) to \$145,931 (90th), median \$76,438

Wholesale and Retail Buyers, Except Farm Products (13-1022.00):

Estimated wage range: \$45,993 (10th) to \$129,558 (90th), median \$72,266

Purchasing Agents, Except Wholesale, Retail, and Farm Products (13-1023.00):

Estimated wage range: \$55,440 (10th) to \$177,780 (90th), median \$96,025

Appraisers of Personal and Business Property (13-2022.00):

Estimated wage range: \$38,198 (10th) to \$98,733 (90th), median \$58,679

Appraisers and Assessors of Real Estate (13-2023.00):

Estimated wage range: \$41,371 (10th) to \$107,635 (90th), median \$64,383

1.4 Verdict

- **Does the model clearly beat the naive baseline? Yes.** RMSE (log-wage): 0.269 vs 0.456 naive. MAE: \$19,586 vs \$31,943 naive — a **38.7% improvement**. Skill profile + job_zone genuinely add predictive value beyond just knowing the percentile level.
- **Does monotonicity hold across all held-out occupations? Yes, perfectly** — 0/173 violations on real test data (not just the synthetic test suite).
- **Do the estimated wage ranges for previously-unmatched occupations look plausible? Yes**, with one caveat worth naming honestly: Real Estate Appraisers (\$64,383 median) and Personal/Business Property Appraisers (\$58,679 median) both land close to their real-world BLS figures. The three buyer/purchasing-agent titles land in a sensible overall range (\$72k-\$96k) but spread wider across similar-sounding titles than expected — a real limitation of interpolating from a 35-dimensional skill-similarity space, worth stating plainly rather than

glossing over.

Phase 1's baseline model is validated. Remaining Phase 1 items: package into `src/` (done — `occupation_features.py`, `wage_model.py`), update README's Method + Limitations sections with these real numbers.